

Glenda Ho Ru Si

Student number: 230572931

Singapore Institute of Management

Programming For Data Science

3 April 2025

Table of Contents

<i>Coursework Project Part 1</i>	3
Part 1a).....	3
Part 1b).....	3
<i>Coursework Project Part 2</i>	5
Data Reading and Cleaning for Part 2a) and 2b)	5
Part 2a).....	5
Part 2b)	7
Part 2c).....	9
<i>References:</i>	11

Coursework Project Part 1

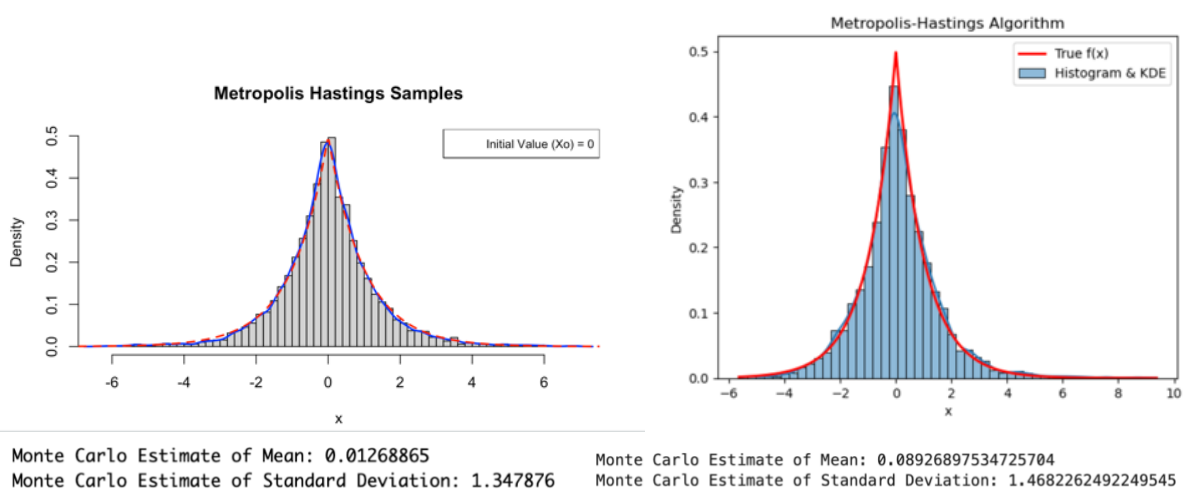
Part 1a)

The aim is to generate random numbers following the given probability density function and evaluate the quality of the sample by comparing its mean and standard deviation.

I begin by defining the probability density function $f(x)$ using the given equation $f(x) = \frac{1}{2} \exp(-|x|)$. Next, I set up the Metropolis-Hastings algorithm to repeat 10000(N), starting at the initial value(X_0) of 0 and standard deviation(s) of 1.

Next, I created a for-loop that iteratively builds up a list of samples. Each iteration proposes a new sample and decides whether to accept it based on the acceptance ratio. If the ratio is greater than the random number(u) generated by a uniform distribution, the newly accepted sample is added to the list of generated samples. If it is not, the list of generated samples remains the same.

After 10000(N) iterations, the newly accepted samples will be added to the histogram, with kernel density plot to give a smooth representation of the data's distribution. The graph of $f(x)$ is superimposed on this figure to illustrate the precision of these approximations. The diagrams below are the generated figures from **R**(left diagram) and **Python**(right diagram).



Part 1b)

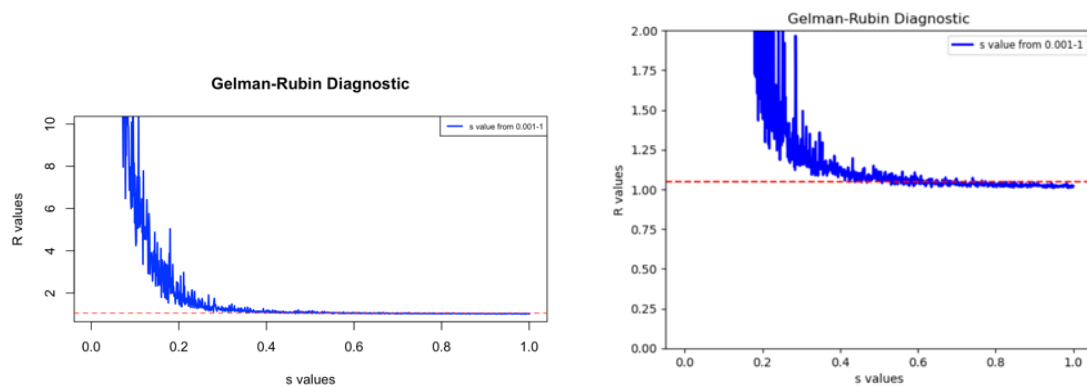
The objective is to assess the convergence of the Metropolis-Hastings algorithm by computing the $R_values(R\hat{)}$ using specified parameters while adjusting the standard deviation (s) within a given range.

First, I define the parameters as 2000(N_1) iterations and the initial values(X_{ob}) as -15,1,15 and 40. The values of the standard deviation(s_values) range from 0.001 to 1.000 in increments of 0.001 and we initialize an array(R_values), with the same length as s_values to store the $R\hat{}$ values for each step size.

Next, I created an outer loop to generate a set of sample(samplesb) from the 4 different initial value(X_{ob}) using the Metropolis-Hastings algorithm. I then created an inner loop to iterate over each value in s_values to calculate the $R_values(R\hat{)}$ for different step sizes and compute the mean and variance of the generated samples and store them in the means and vars arrays at the corresponding index J .

Next, I calculated the $R_values(R\hat{)}$ using the overall mean(M), the overall within sample variance(W), the between sample variance(B) and the within sample variance(w_in).

After computing the $R_values(R\hat{ })$, I plot the R_values against the s_values . The diagram helps to assess convergence by showing whether $R_values(R\hat{ })$ fall below the threshold of 1.05 across the range of step sizes. A R_value near 1 suggests convergence and a R_value above 1.05 suggests the lack of convergence. The diagrams below are the generated figures from **R**(left diagram) and **Python**(right diagram).



R hat value: 1.013815

R hat value: 1.0212967205942125

Coursework Project Part 2

In R, I explicitly ensured that all required packages are installed and loaded without triggering unnecessary reinstallation or an R session restart due to large number of packages required. If any packages are missing, only those are installed, avoiding redundant installations. This approach optimizes workflow by preventing disruptions while ensuring that all dependencies are met before executing further analyses.

```
{r}
packages <- c("reshape2", "ggthemes", "viridis", "RColorBrewer",
             "lubridate", "car", "caret", "glmnet", "tidyverse")

install_missing <- packages[!(packages %in% installed.packages()[,
"Package"])]
if(length(install_missing)) install.packages(install_missing)
```

Example: `lapply(packages, library, character.only = TRUE)`

Data Reading and Cleaning for Part 2a) and 2b)

Due to the huge dataset and the constraints of my computer, I began by preprocessing the CSV file. I used a for-loop to iterate through the data for each year from 1995 to 1999. During this process, I handled missing or invalid values and selected key columns such as "Year," "Tailnum," "CRSDepTime," and others crucial for my evaluation.

Example:

```
Columns_Needed = ["Year", "DayOfWeek", "TailNum", "ArrDelay", "DepDelay", "CRSDepTime", "DepTime"]
```

After cleaning the data for each year, I saved it as a separate CSV file. To integrate the processed data, I utilized an additional for-loop to read each CSV file and combine them into a singular primary data frame covering the years 1995 to 1999. This consolidated dataset will be utilized for questions 2a and 2b. For ease of use, I included a prompt at the end of the code to ask whether the newly created CSV file should be deleted. If the user chooses, the file can be automatically removed; otherwise, it will be retained.

Part 2a)

The aim is to select a consecutive 5-year subset(1995 to 1999) from the dataset and determine the optimal days of the week and times of the day each year that experience the least delays.

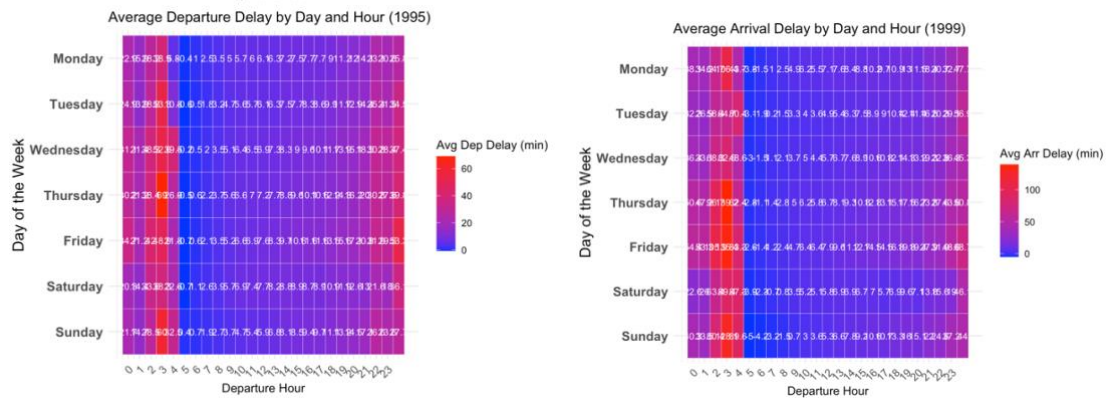
To analyze flight delays across different days and times, I extracted data from multiple annual records, focusing on key columns such as "Year," "DepTime," "DepDelay," and "ArrDelay." After filtering out missing values, I converted departure times into hourly intervals, allowing for a structured comparison of delays by time of day and day of the week.

Next, I introduced a new column, "DepHour," which categorized flights based on their scheduled departure hour, facilitating time-based analysis. To ensure accuracy, I also mapped numerical day values to their corresponding weekday names, making the data more interpretable.

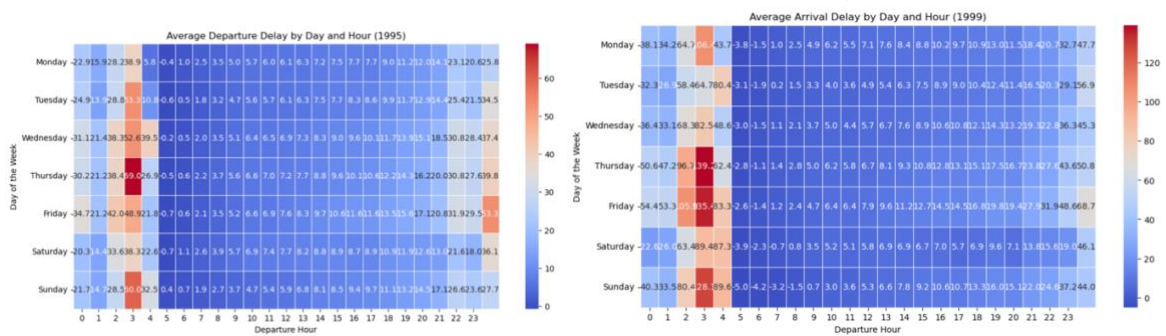
Following this, I calculated average departure and arrival delays for each hour and weekday, grouping the results by year. The data was then reshaped into a pivot table format, creating a structured view of delay patterns over time. This allowed for easier visualization and comparison across different time slots.

Finally, I generated heatmaps to display average departure and arrival delays across various hours and weekdays, using a color gradient where blue indicated minimal delays and red signified longer delays. These visualizations helped highlight patterns, such as peak delay periods and days with consistently lower delays. By comparing results across years, I identified trends and variations in delay patterns, offering insights into the best times to travel with minimal disruptions. The diagrams below are the generated figures.

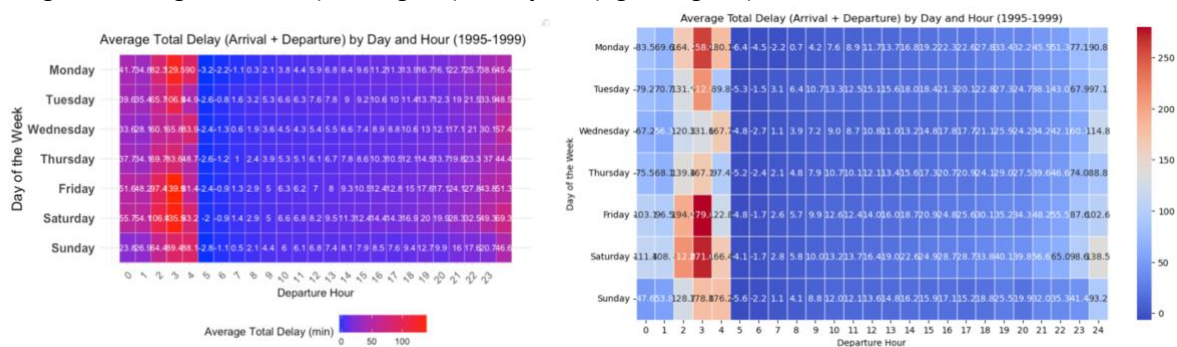
R programming



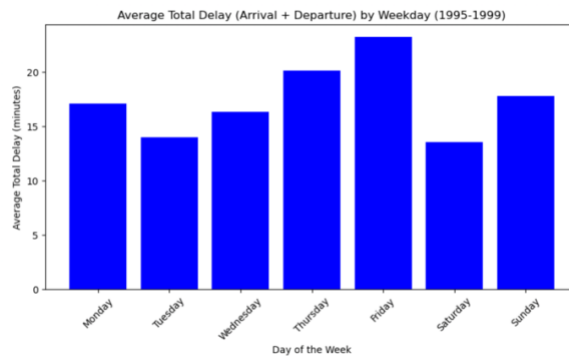
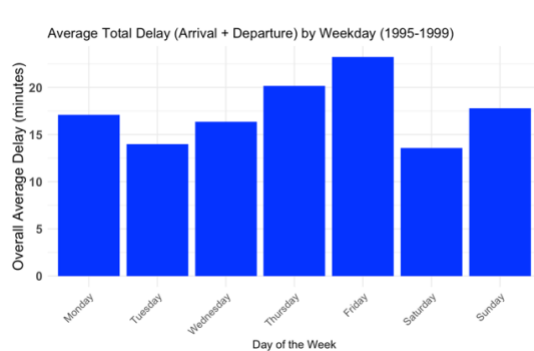
Python Programming



To further enhance the visualization, I generated a heatmap displaying the average total delay(Arrival + Departure) by day and hour for the years 1995-1999. The accompanying bar chart highlights that the boxes between 4:30 AM and 5:30 AM have the darkest blue color, signifying the least amount of delay during this time window. This suggests that early morning flights within this timeframe experience the shortest delays, making it the most optimal period to minimize delays. The analysis provides valuable insights into when delays are typically at their lowest, aiding in scheduling decisions for more timely departures. The diagrams below are the generated figures from **R**(left diagram) and **Python**(right diagram).



To determine the average total delay(Arrival + Departure) by weekday for the years 1995-1999, I generated a bar chart displaying the average total delay(Arrival + Departure) in minutes for each weekday. The analysis reveals that Saturday has the least amount of delay, with an average arrival delay of approximately 14 minutes, making it the best day for minimizing delays. The diagrams below show the generated figures from **R** (on the left) and **Python** (on the right). These visualizations allow for a clear comparison between the two tools and provide insights into how delays vary across different weekdays. The R chart offers one perspective, while the Python chart provides another, both confirming the trend of minimal delays on Saturdays.



In conclusion, the optimal days of the week and times of the day each year that experience the least delays is between 4.30am to 5.30am on Saturday.

Part 2b)

The aim is to analyse whether older aircraft experience increasing delays over the years.

First, I reload the dataset from above and verify that the consecutive 5 year subset is correct.

```
df = pd.concat([Empty_dictQ2ab.values(), ignore_index=True])
pdf1 = pd.read_csv("plane-data.csv")
print("Years in reloaded dataset:", df['Year'].unique())
```

Example: Years in reloaded dataset: [1995 1996 1997 1998 1999]

To analyze the impact of aircraft age on flight delays, I created a new data frame, "oldplanedelay," by extracting key columns such as "Year," "DepDelay," "ArrDelay," and "TailNum" from the main dataset. I introduced a new column, "TotalDelay," by summing both departure and arrival delays, allowing for a comprehensive measure of total delay per flight. To ensure accuracy, only rows where "TotalDelay" was non-negative were retained, eliminating instances where early arrivals could skew the analysis.

Next, I constructed another data frame, "planedata," by extracting aircraft tail numbers and their corresponding issue dates from "pdf1." Rows with missing "issue_date" values were removed to maintain data integrity. This ensured that every aircraft included in the analysis had a known manufacturing date, facilitating a reliable comparison between aircraft age and delays.

Finally, both data frames were printed to the console for inspection, verifying the correctness of data processing. The generated visualizations illustrate the findings, showcasing potential patterns between aircraft age and flight delays. The diagrams below are the generated figures.

R programming

```
print(planedata)
## # A tibble: 4,480 x 2
##   tailnum issue_date
##   <chr>   <chr>
## 1 N10156 02/13/2004
## 2 N102UW 05/26/1999
## 3 N10323 07/01/1997
## 4 N103US 06/18/1999
## 5 N104UA 01/26/1998
## 6 N104UW 07/02/1999
## 7 N10575 06/24/2003
## 8 N105UA 10/15/2007
## 9 N105UW 07/22/1999
## 10 N106US 08/05/1999
## # i 4,470 more rows

print(oldplanedelay)
## # A tibble: 14,775,229 x 3
##   Year TotalDelay TailNum
##   <dbl>   <dbl>   <chr>
## 1 1995      27 N7298U
## 2 1995       4 N7449U
## 3 1995      15 N7257U
## 4 1995       0 N7282U
## 5 1995       1 N7270U
## 6 1995       7 N7279U
## 7 1995      65 N7448U
## 8 1995      19 N7291U
## 9 1995      15 N7276U
## 10 1995      26 N7295U
## # i 14,775,219 more rows
```

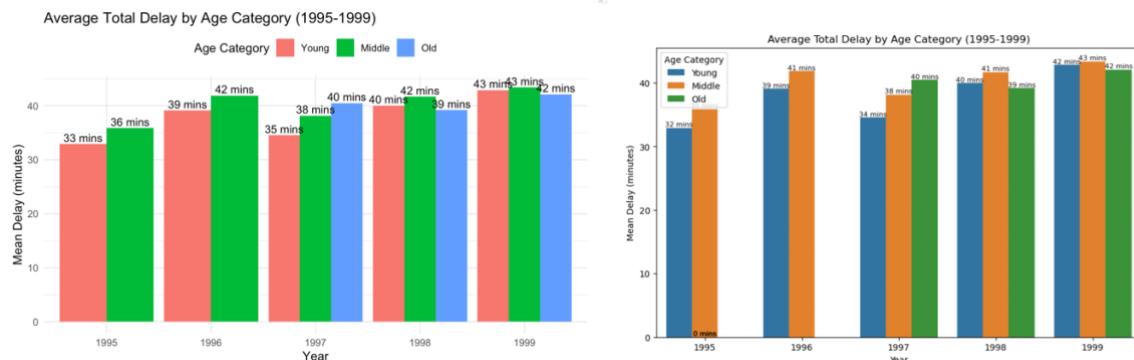
Python Programming

tailnum	issue_date	Year	TotalDelay	TailNum
34 N10156	02/13/2004	1995	27.0	N7298U
35 N102UW	05/26/1999	1995	4.0	N7449U
36 N10323	07/01/1997	1995	15.0	N7257U
37 N103US	06/18/1999	1995	0.0	N7282U
38 N104UA	01/26/1998	1995	1.0	N7270U
...	...	...	...	...
5024 N997DL	03/11/1992	1999	1.0	N2BFAA
5025 N998AT	01/23/2003	1999	5.0	N2AGAA
5026 N998DL	04/02/1992	1999	85.0	N885AA
5027 N999CA	07/09/2008	1999	0.0	N2DAAA
5028 N999DN	04/02/1992	1999	5.0	N2BAAA

Next, I converted the "issue_date" column in "planedata" to a "datetime" format and extracts the year into a new column, "ManufactureYear". Then, merge "planedata" with "oldplanedelay" based on the "TailNum" and

“tailnum” columns, using a left join to include the “ManufactureYear” in the “oldplanedelay” DataFrame. A new column, “AircraftAge”, is calculated by subtracting the “ManufactureYear” from the Year of the flight. Next, the code drops rows with missing “AircraftAge” values and filters out any rows where “AircraftAge is negative”, ensuring only valid data is kept.

Next, I categorized the aircraft into age groups based on their “AircraftAge”. The age groups are defined as "Young" (0-10 years), "Middle" (10-20 years), and "Old" (20+ years). Then, I calculated the average “TotalDelay” for each Year and “AgeCategory”. I generated a bar graph to visualise the average total delay by age groups to aid visualization. The diagrams below are the generated figures from **R**(left diagram) and **Python**(right diagram).



For better illustration, I determined the age category with the most significant delay each year. This approach allows for a clearer comparison to assess whether older planes lead to greater delays. The diagrams below show the generated figures from **R** (on the left) and **Python** (on the right).

Year 1995: Middle has the highest average total delay of 35:53.	Year 1995: Middle has the highest average total delay of 35:52.
Year 1996: Middle has the highest average total delay of 41:53.	Year 1996: Middle has the highest average total delay of 41:52.
Year 1997: Old has the highest average total delay of 40:28.	Year 1997: Old has the highest average total delay of 40:27.
Year 1998: Middle has the highest average total delay of 41:42.	Year 1998: Middle has the highest average total delay of 41:41.
Year 1999: Middle has the highest average total delay of 43:23.	Year 1999: Middle has the highest average total delay of 43:23.

As observed, the "Middle" age category (10-20 years old) consistently had the average total delay, with delays increasing each year. In 1997, however, the "Old" age category (over 20 years old) had the highest delay. This suggests that while older aircraft may sometimes face more delays, the middle-aged fleet generally experiences the highest delays during this period.

Purely out of interest, I performed a Two-Way ANOVA test on my dataset to analyse the impact of Year, Age Category, and their interaction on total delay. The goal was to determine whether average total delay varied significantly across these factors.

First, we tested whether average total delay differed across years at a significance level of 0.05. The results indicate a significant effect of Year, with $F(4, 3017647) = 1872.877$, p-value is less than $2.2e-16$. Since this value is significantly below 0.05, we reject the null hypothesis (H_0) and conclude that there is strong evidence that mean delay significantly differs across years. This suggests that external factors influencing delays may vary over time.

Next, we examined whether mean delay differed across age categories at the same significance level. The F-statistic for “AgeCategory” was $F(2, 3017647) = 95.413$, p-value $< 2.2e-16$, which also falls below the 0.05 threshold. Thus, we reject the null hypothesis and conclude that aircraft age category has a significant effect on total delay. The diagrams below are the generated figures from **R**(left diagram) and **Python**(right diagram).

```
## Anova Table (Type II tests)
##
## Response: TotalDelay
##               Sum Sq   Df F value    Pr(>F)
## factor(AgeCategory) 9.2175e+05    2   95.413 < 2.2e-16 ***
## factor(Year)        3.6186e+07    4 1872.877 < 2.2e-16 ***
## Residuals          1.4576e+10 3017647
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

               sum_sq      df      F      PR(>F)
C(AgeCategory) 9.217484e+05    2.0   95.413278 3.663111e-42
C(Year)        3.618618e+07    4.0 1872.876719 0.000000e+00
Residual      1.457612e+10 3017647.0      NaN      NaN
```

The significant result for “AgeCategory” suggests that older planes (or planes in the "Old" category) have different (likely higher) delays compared to younger planes.

In conclusion, there is no consistent pattern indicating that older planes experience more delays each year. Instead, middle-aged planes (10-20 years) tend to experience the highest delays in most years, except for 1997, where older planes had the highest delay. While aircraft age significantly impacts delays, other external factors (e.g., airline policies, maintenance cycles, air traffic conditions) likely play a role in shaping these trends over time.

Part 2c)

The aim is to fit a logistic regression model each year to predict the probability of diverted US flights. The model will incorporate as many features as possible, including departure date attributes, scheduled departure and arrival times, airport coordinates, distance between departure and planned arrival locations, and the carrier. The coefficients will be visualized across different years.

Due to the large size of the file and the number of loops in the original code, I decided to simplify the process for Part 2c. Instead of implementing a loop to read the CSV files for each year individually (1995-1999), I opted to read all the relevant data at once. I then selected only the necessary columns, such as “Year”, “Month”, “DayofMonth”, etc., which reduced memory usage and streamlined the data processing. This approach eliminated the need for multiple loops, allowing for faster data manipulation and analysis, while still ensuring accurate results.

Example:

```
Columns_Needed <- c("Month", "DayofMonth", "CRSDepTime", "CRSArrTime", "Distance",
                    "UniqueCarrier", "Origin", "Dest", "Year", "Diverted")
```

I processed flight data from 1995 to 1999 to predict whether a flight will be diverted, based on several factors. For each year, the data is first checked to ensure all required information is present. If any essential data is missing, that year is skipped. The dataset is cleaned by removing any incomplete records and converting non-numeric information (like the month, carrier, or airport) into a format that can be used by the model. It also removes any features that do not vary across the data, improving the model's ability to identify meaningful patterns.

The data is then split into two sets: one for training the model and one for testing it. Before training, the training data is adjusted so that all features are on the same scale, ensuring fair comparisons between them. A logistic regression model, which is used to predict outcomes based on various input factors, is then trained using the cleaned, scaled data. The model's performance is assessed by examining the coefficients, which indicate the strength and direction of the relationship between each factor and the likelihood of a flight being diverted. This process is repeated each year, allowing for a year-by-year analysis of which factors influence flight diversion and providing insights into how these factors have changed over time. The diagrams below show the generated figures from **R** (on the left) and **Python** (on the right). Due to space constraint, only a portion will be displayed.

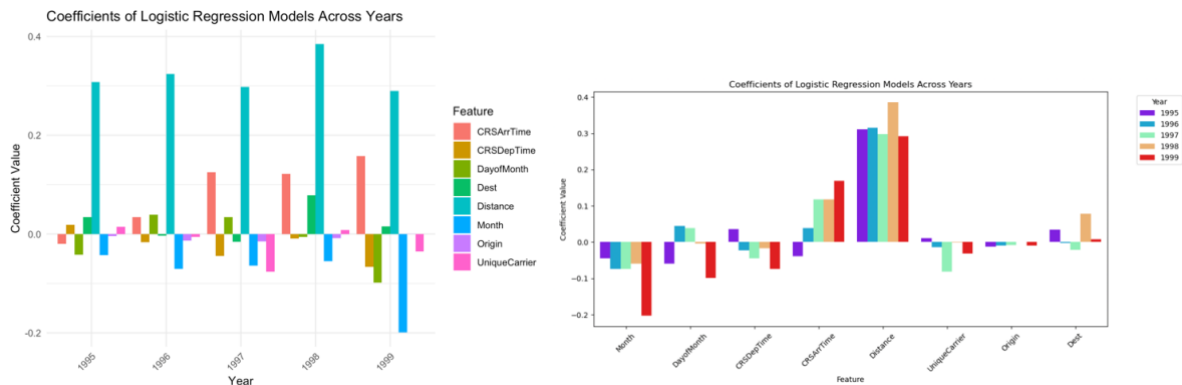
```
## Coefficients for 1995 :      Coefficients for 1995:
## Month : -0.04278626      Month: -0.0450318292282728
## DayofMonth : -0.0414658  DayofMonth: -0.0588225123186723
## CRSDepTime : 0.01849593  CRSDepTime: 0.036062500121846024
## CRSArrTime : -0.01953622 CRSArrTime: -0.038997831257622254
## Distance : 0.3080907     Distance: 0.31083119462901165
## UniqueCarrier : 0.01446994 UniqueCarrier: 0.010445504088862695
## Origin : -0.004013094    Origin: -0.012236528575051121
## Dest : 0.03446517       Dest: 0.0337447242518104
##
## Coefficients for 1996 :      Coefficients for 1996:
## Month : -0.07050941      Month: -0.07405568476209767
## DayofMonth : 0.03902386  DayofMonth: 0.043843644450481216
## CRSDepTime : -0.01661941 CRSDepTime: -0.0224786326003319
## CRSArrTime : 0.03451863  CRSArrTime: 0.03888314635374962
## Distance : 0.3239862     Distance: 0.3150405201501788
## UniqueCarrier : -0.005790866 UniqueCarrier: -0.013386882086998089
## Origin : -0.01291547     Origin: -0.010306599615930669
## Dest : -0.003494068     Dest: -0.002879645425197614
```

The coefficient interpretations remain consistent from 1995 to 1999, as illustrated in the bar chart below.

$$\text{Logit}(y) = \beta_0 + \beta_1 \times \text{Month} + \beta_2 \times \text{DayofMonth} + \beta_3 \times \text{CRSDepTime} + \beta_4 \times \text{CRSArrTime} + \beta_5 \times \text{Distance} + \beta_6 \times \text{UniqueCarrier} + \beta_7 \times \text{Origin} + \beta_8 \times \text{Dest}$$

Where, $\text{Logit}(y) = \log - \text{odds of target variable } y \text{ (Diverted)}$
 $\beta_0 = \text{Intercept}$
 $\beta_{1-8} = \text{Coefficient obtained from logistic regression}$
 $\text{Month to Dest} = \text{Corresponding features}$

In the regression model, the coefficients indicate the effect size, representing how each feature influences the logarithm of the odds ratio for a flight being diverted. A larger coefficient signifies a stronger impact on diversion probability, assuming all other factors remain constant. The coefficient's sign determines the direction of the effect: a positive value implies that an increase in the feature raises the log-odds of diversion, whereas a negative value suggests it lowers them. Once identified, the coefficients are stored in a pre-existing list categorized by year. To enhance visualization, a bar chart is used, as shown below. The diagrams below show the generated figures from **R** (on the left) and **Python** (on the right).



The use of two different types of plots improves data visualization and clarity. Both graphs visualize the coefficients of logistic regression models across multiple years, but they emphasize different perspectives. The R graph, where the x-axis represents the year and the features are distinguished by color, highlights how the significance of each feature changes over time. This allows for an analysis of trends, showing whether certain factors, such as departure time or distance, became more or less influential in predicting delays from 1995 to 1999. In contrast, the Python graph, which places features on the x-axis and represents different years through color, focuses on comparing the impact of each feature across different years. This format makes it easier to identify which features consistently influence delays and which vary significantly over time. While the R graph is useful for examining trends and shifts in predictive power, the Python graph provides a clearer comparison of feature importance within each year. These insights help in comprehending the factors influencing flight diversions and how different features impact them over time.

References:

GeeksforGeeks. (n.d.). *Create heatmap in R using ggplot2*. Retrieved from <https://www.geeksforgeeks.org/create-heatmap-in-r-using-ggplot2/>

Seaborn. (n.d.). *seaborn.heatmap*. Retrieved from <https://seaborn.pydata.org/generated/seaborn.heatmap.html>

Insijets. (n.d.). *Does the age of an aircraft affect its safety?* Retrieved from <https://insijets.com/blog/does-the-age-of-an-aircraft-affect-its-safety>

Brownlee, J. (2022). *Train-test split for evaluating machine learning algorithms*. Machine Learning Mastery. Retrieved from <https://machinelearningmastery.com/train-test-split-for-evaluating-machine-learning-algorithms/>